Laboratory work 8

Views and Roles (PostgreSQL)

Student: Galymm Orunby
Group: Tuesday 13.00-14.00

Part 1. VIEW

Task 1

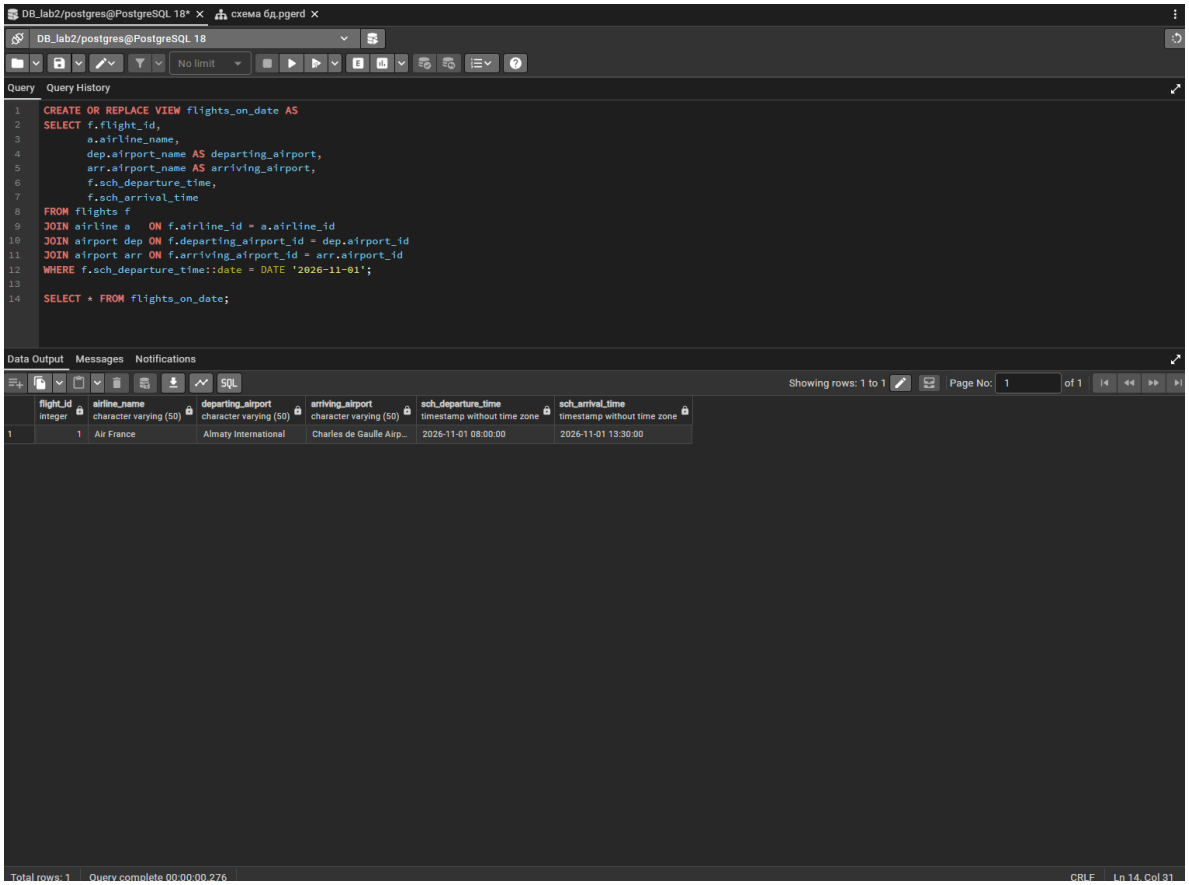
Create a view to show details of all flights that are departing on a specific date.

SQL code:

```
CREATE OR REPLACE VIEW flights_on_date AS
SELECT f.flight_id,
       a.airline_name,
       dep.airport_name AS departing_airport,
       arr.airport_name AS arriving_airport,
       f.sch_departure_time,
       f.sch_arrival_time
FROM flights f
JOIN airline a   ON f.airline_id = a.airline_id
JOIN airport dep ON f.departing_airport_id = dep.airport_id
JOIN airport arr ON f.arriving_airport_id = arr.airport_id
WHERE f.sch_departure_time::date = DATE '2026-11-01';

SELECT * FROM flights_on_date;
```

Screenshot:



Task 2

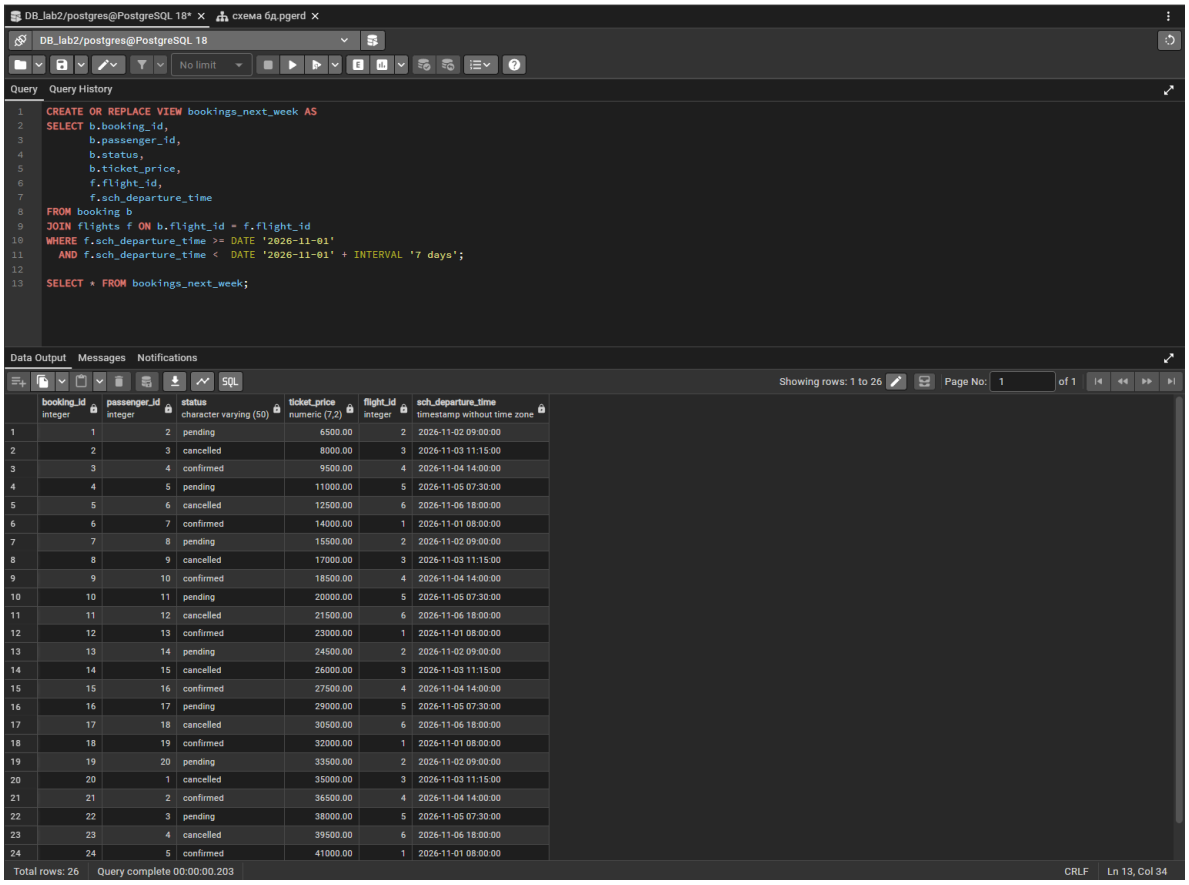
Create a view that shows bookings for flights scheduled to depart within the next week.

SQL code:

```
CREATE OR REPLACE VIEW bookings_next_week AS
SELECT b.booking_id,
       b.passenger_id,
       b.status,
       b.ticket_price,
       f.flight_id,
       f.sch_departure_time
FROM booking b
JOIN flights f ON b.flight_id = f.flight_id
WHERE f.sch_departure_time >= DATE '2026-11-01'
  AND f.sch_departure_time <  DATE '2026-11-01' + INTERVAL '7 days';

SELECT * FROM bookings_next_week;
```

Screenshot:



DB_lab2/postgres@PostgreSQL 18* | схема бд.pgerd

DB_lab2/postgres@PostgreSQL 18

Query | Query History

```
CREATE OR REPLACE VIEW bookings_next_week AS
SELECT b.booking_id,
       b.passenger_id,
       b.status,
       b.ticket_price,
       f.flight_id,
       f.sch_departure_time
FROM booking b
JOIN flights f ON b.flight_id = f.flight_id
WHERE f.sch_departure_time >= DATE '2026-11-01'
  AND f.sch_departure_time <  DATE '2026-11-01' + INTERVAL '7 days';

SELECT * FROM bookings_next_week;
```

Data Output | Messages | Notifications

Showing rows: 1 to 26 | Page No: 1 of 1

	booking_id integer	passenger_id integer	status character varying (50)	ticket_price numeric (7,2)	flight_id integer	sch_departure_time timestamp without time zone
1	1	2	pending	6500.00	2	2026-11-02 09:00:00
2	2	3	cancelled	8000.00	3	2026-11-03 11:15:00
3	3	4	confirmed	9500.00	4	2026-11-04 14:00:00
4	4	5	pending	11000.00	5	2026-11-05 07:30:00
5	5	6	cancelled	12500.00	6	2026-11-06 18:00:00
6	6	7	confirmed	14000.00	1	2026-11-01 08:00:00
7	7	8	pending	15500.00	2	2026-11-02 09:00:00
8	8	9	cancelled	17000.00	3	2026-11-03 11:15:00
9	9	10	confirmed	18500.00	4	2026-11-04 14:00:00
10	10	11	pending	20000.00	5	2026-11-05 07:30:00
11	11	12	cancelled	21500.00	6	2026-11-06 18:00:00
12	12	13	confirmed	23000.00	1	2026-11-01 08:00:00
13	13	14	pending	24500.00	2	2026-11-02 09:00:00
14	14	15	cancelled	26000.00	3	2026-11-03 11:15:00
15	15	16	confirmed	27500.00	4	2026-11-04 14:00:00
16	16	17	pending	29000.00	5	2026-11-05 07:30:00
17	17	18	cancelled	30500.00	6	2026-11-06 18:00:00
18	18	19	confirmed	32000.00	1	2026-11-01 08:00:00
19	19	20	pending	33500.00	2	2026-11-02 09:00:00
20	20	1	cancelled	35000.00	3	2026-11-03 11:15:00
21	21	2	confirmed	36500.00	4	2026-11-04 14:00:00
22	22	3	pending	38000.00	5	2026-11-05 07:30:00
23	23	4	cancelled	39500.00	6	2026-11-06 18:00:00
24	24	5	confirmed	41000.00	1	2026-11-01 08:00:00

Total rows: 26 | Query complete 00:00:00.203 | CRLF | Ln 13, Col 34

Task 3

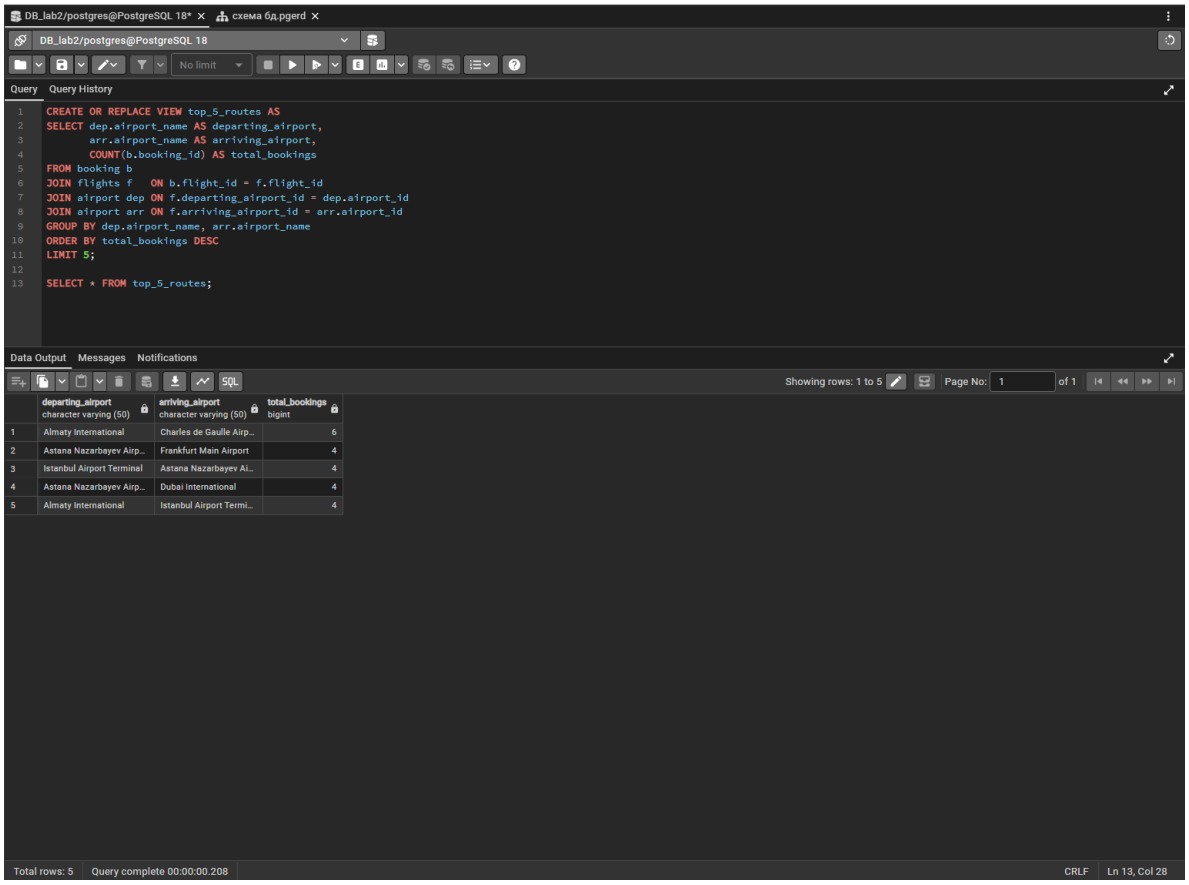
Create a view to show the top 5 most popular flight routes based on the number of bookings.

SQL code:

```
CREATE OR REPLACE VIEW top_5_routes AS
SELECT dep.airport_name AS departing_airport,
       arr.airport_name AS arriving_airport,
       COUNT(b.booking_id) AS total_bookings
FROM booking b
JOIN flights f   ON b.flight_id = f.flight_id
JOIN airport dep ON f.departing_airport_id = dep.airport_id
JOIN airport arr ON f.arriving_airport_id = arr.airport_id
GROUP BY dep.airport_name, arr.airport_name
ORDER BY total_bookings DESC
LIMIT 5;

SELECT * FROM top_5_routes;
```

Screenshot:



Task 4

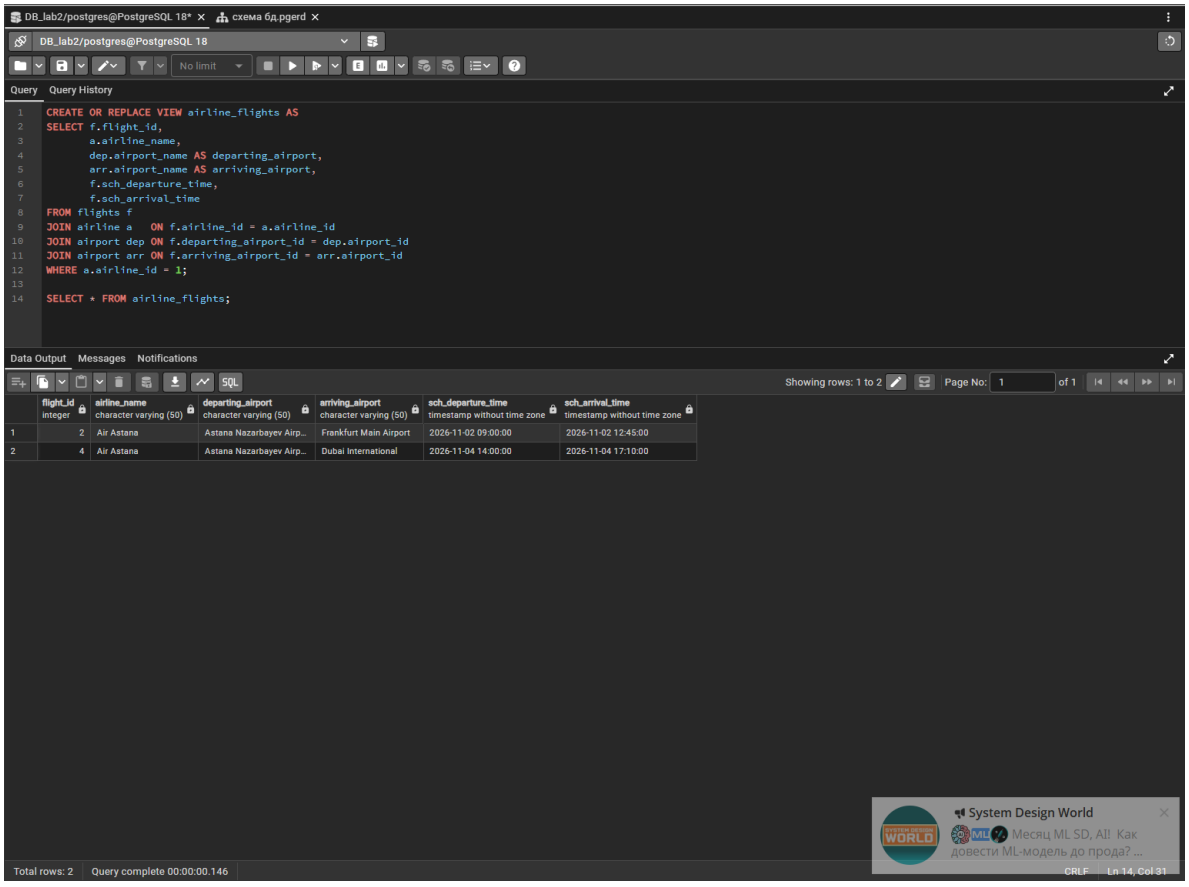
Create a view that lists all flights for a specific airline.

SQL code:

```
CREATE OR REPLACE VIEW airline_flights AS
SELECT f.flight_id,
       a.airline_name,
       dep.airport_name AS departing_airport,
       arr.airport_name AS arriving_airport,
       f.sch_departure_time,
       f.sch_arrival_time
FROM flights f
JOIN airline a   ON f.airline_id = a.airline_id
JOIN airport dep ON f.departing_airport_id = dep.airport_id
JOIN airport arr ON f.arriving_airport_id = arr.airport_id
WHERE a.airline_id = 1;

SELECT * FROM airline_flights;
```

Screenshot:



Task 5

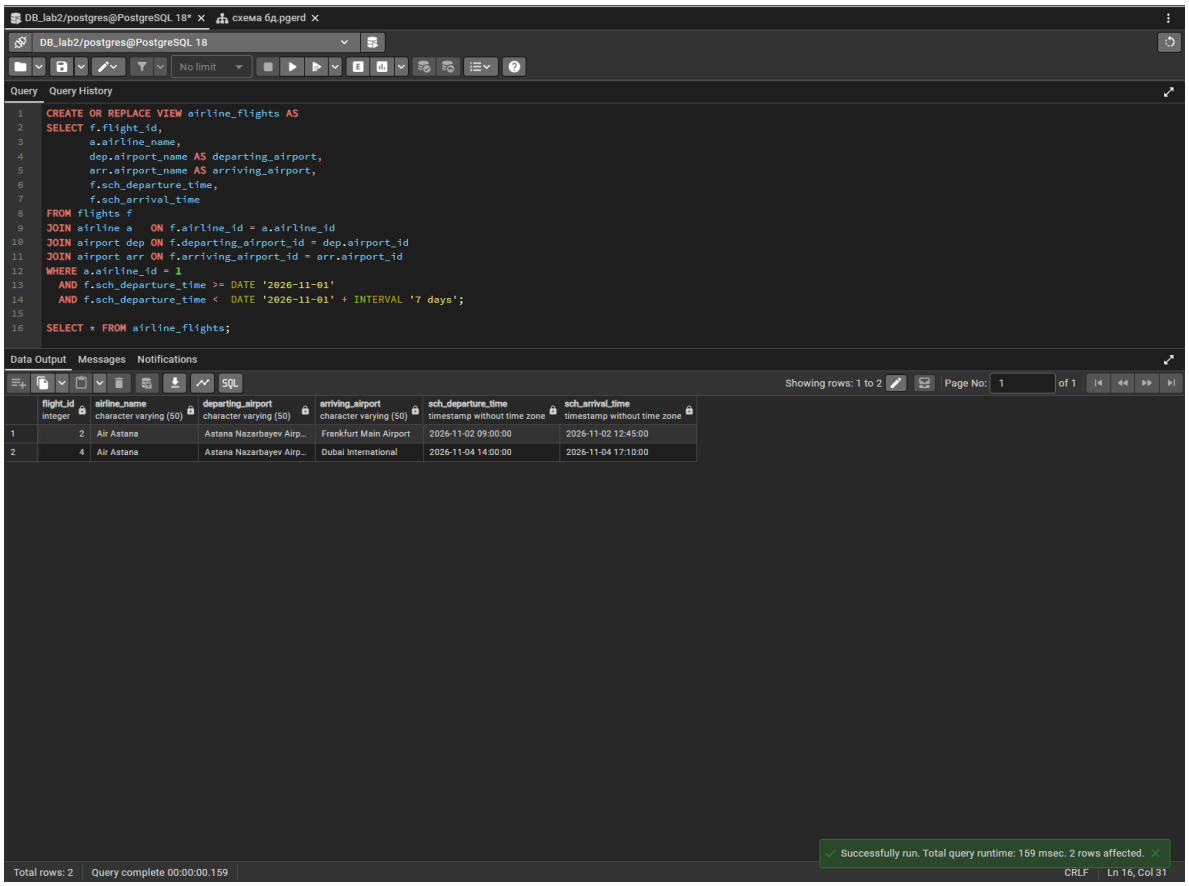
Modify the view created in task 4 to show only flights departing within the next 7 days for a specific airline.

SQL code:

```
CREATE OR REPLACE VIEW airline_flights AS
SELECT f.flight_id,
       a.airline_name,
       dep.airport_name AS departing_airport,
       arr.airport_name AS arriving_airport,
       f.sch_departure_time,
       f.sch_arrival_time
FROM flights f
JOIN airline a   ON f.airline_id = a.airline_id
JOIN airport dep ON f.departing_airport_id = dep.airport_id
JOIN airport arr ON f.arriving_airport_id = arr.airport_id
WHERE a.airline_id = 1
  AND f.sch_departure_time >= DATE '2026-11-01'
  AND f.sch_departure_time <  DATE '2026-11-01' + INTERVAL '7 days';

SELECT * FROM airline_flights;
```

Screenshot:



Task 6

Create a view to show flights that are delayed by more than 24 hours.

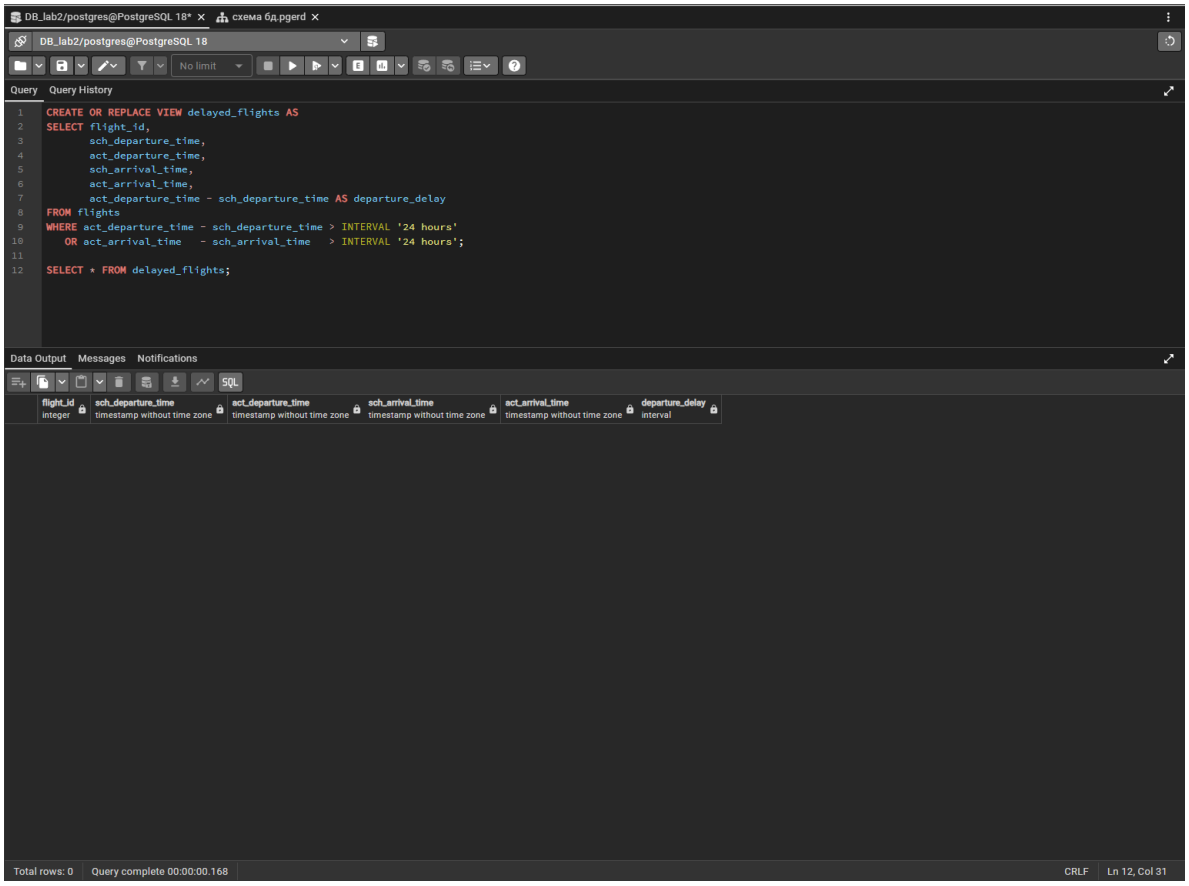
SQL code:

```
CREATE OR REPLACE VIEW delayed_flights AS
SELECT flight_id,
       sch_departure_time,
       act_departure_time,
       sch_arrival_time,
       act_arrival_time,
       act_departure_time - sch_departure_time AS departure_delay
FROM flights
WHERE act_departure_time - sch_departure_time > INTERVAL '24 hours'
   OR act_arrival_time   - sch_arrival_time   > INTERVAL '24 hours';

SELECT * FROM delayed_flights;
```

Note: the result is empty because there are no flights in the database delayed by more than 24 hours.

Screenshot:



Task 7

Create a view in which you can display the full name and country of origin of passengers who made bookings on Leffler-Thompson platform. Then show the list of those passengers.

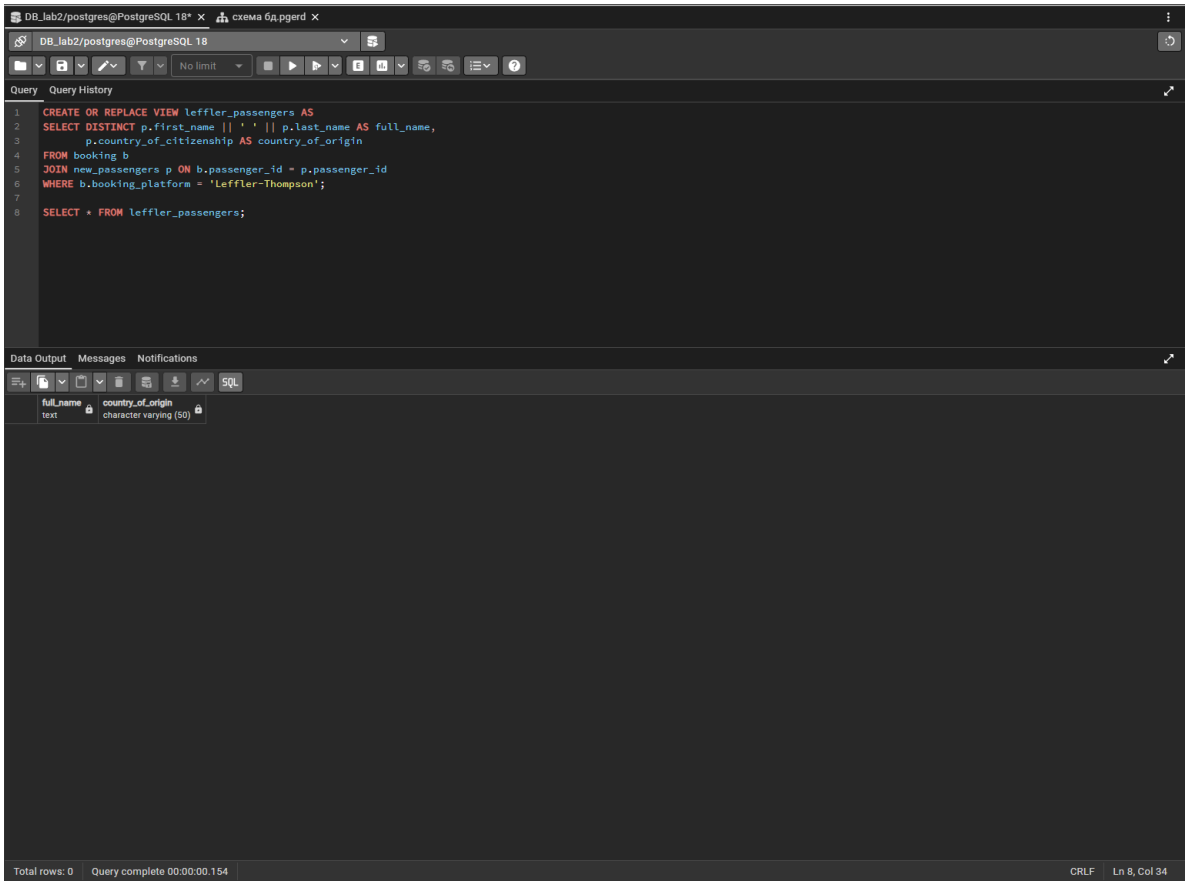
SQL code:

```
CREATE OR REPLACE VIEW leffler_passengers AS
SELECT DISTINCT p.first_name || ' ' || p.last_name AS full_name,
       p.country_of_citizenship AS country_of_origin
FROM booking b
JOIN new_passengers p ON b.passenger_id = p.passenger_id
WHERE b.booking_platform = 'Leffler-Thompson';

SELECT * FROM leffler_passengers;
```

Note: the result is empty because the booking_platform column contains only the values app, agent, kiosk and web; there are no bookings from Leffler-Thompson in the data.

Screenshot:



Task 8

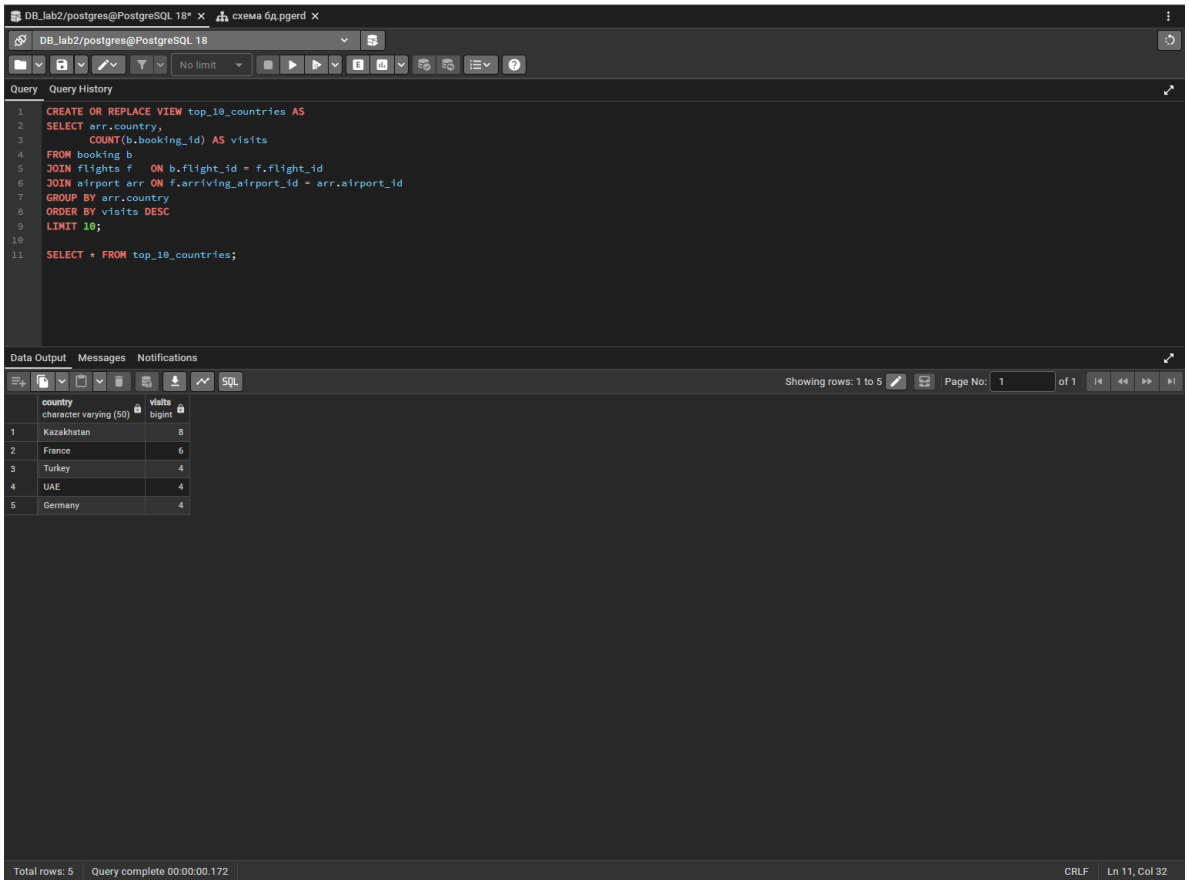
Create a view that shows top 10 most visited countries.

SQL code:

```
CREATE OR REPLACE VIEW top_10_countries AS
SELECT arr.country,
       COUNT(b.booking_id) AS visits
FROM booking b
JOIN flights f   ON b.flight_id = f.flight_id
JOIN airport arr ON f.arriving_airport_id = arr.airport_id
GROUP BY arr.country
ORDER BY visits DESC
LIMIT 10;

SELECT * FROM top_10_countries;
```

Screenshot:



Task 9

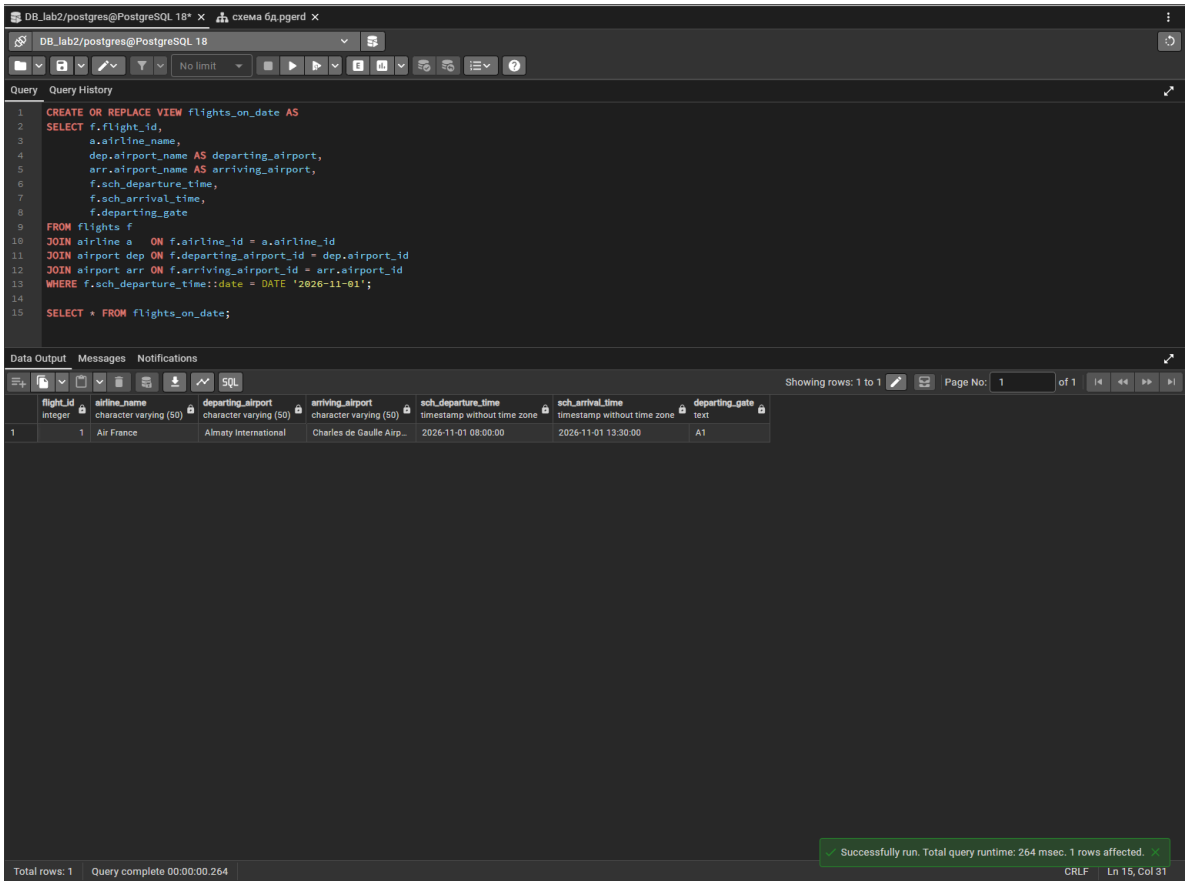
Update any of the created views by adding new information in the view table. Show results.

SQL code:

```
CREATE OR REPLACE VIEW flights_on_date AS
SELECT f.flight_id,
       a.airline_name,
       dep.airport_name AS departing_airport,
       arr.airport_name AS arriving_airport,
       f.sch_departure_time,
       f.sch_arrival_time,
       f.departing_gate
FROM flights f
JOIN airline a   ON f.airline_id = a.airline_id
JOIN airport dep ON f.departing_airport_id = dep.airport_id
JOIN airport arr ON f.arriving_airport_id = arr.airport_id
WHERE f.sch_departure_time::date = DATE '2026-11-01';

SELECT * FROM flights_on_date;
```

Screenshot:



Task 10

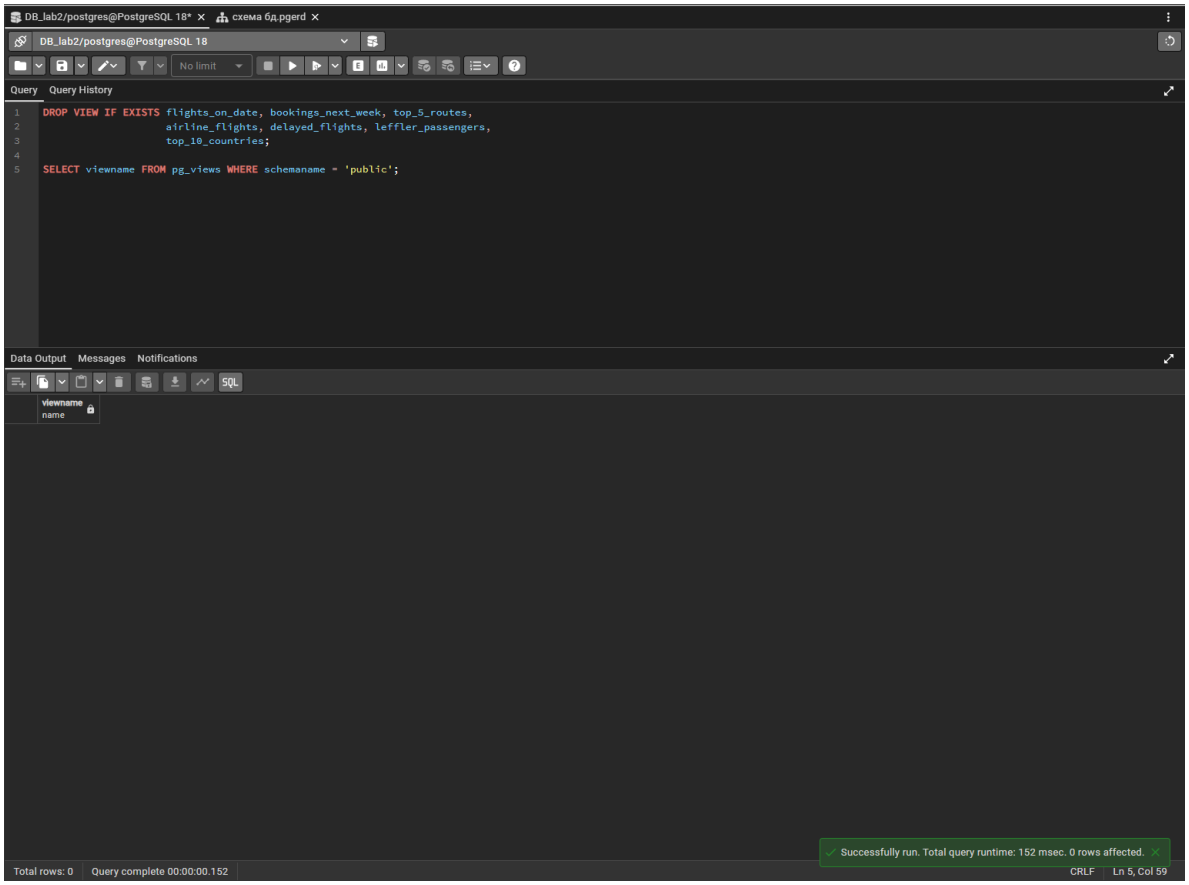
Drop all existing views.

SQL code:

```
DROP VIEW IF EXISTS flights_on_date, bookings_next_week, top_5_routes,
                    airline_flights, delayed_flights, leffler_passengers,
                    top_10_countries;

SELECT viewname FROM pg_views WHERE schemaname = 'public';
```

Screenshot:



Part 2. ROLES

Task 11

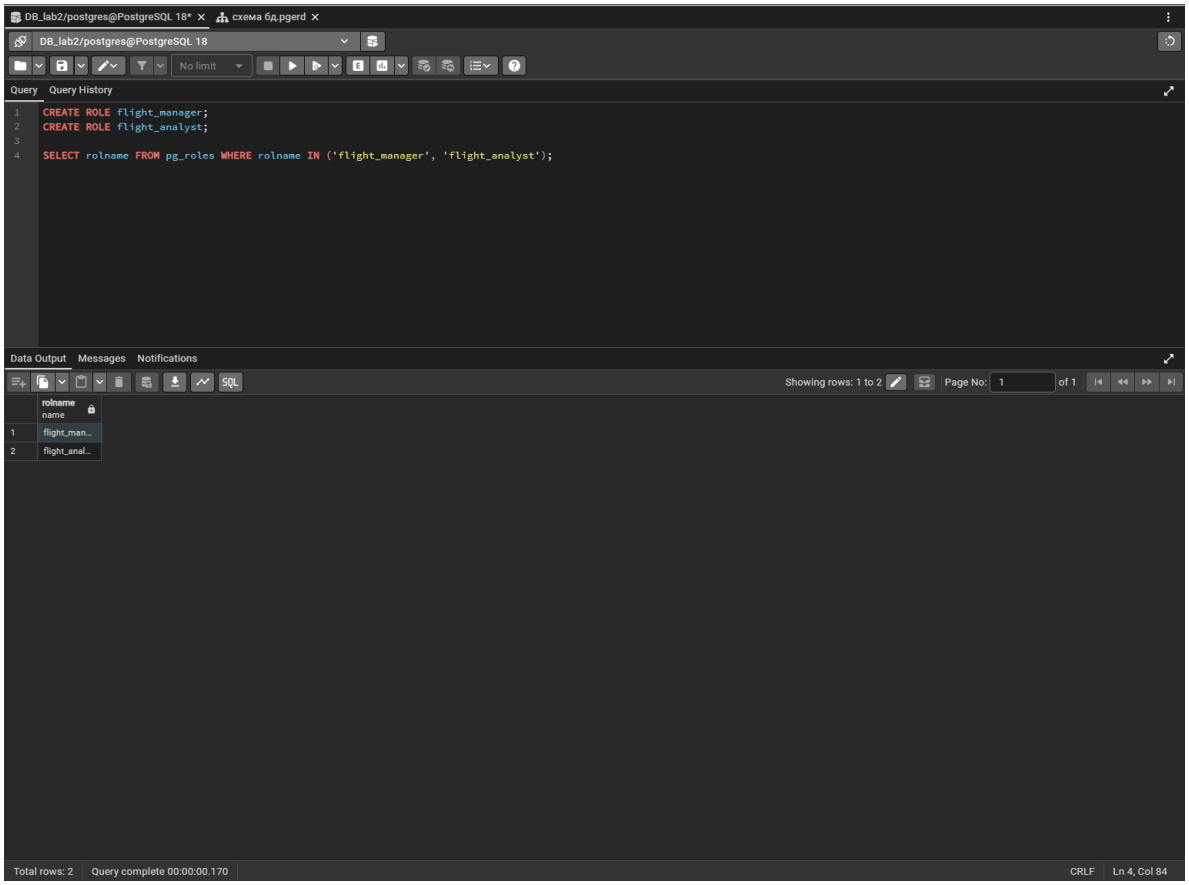
Create the group roles flight_manager and flight_analyst, then show the list of roles.

SQL code:

```
CREATE ROLE flight_manager;
CREATE ROLE flight_analyst;

SELECT rolname FROM pg_roles WHERE rolname IN ('flight_manager', 'flight_analyst');
```

Screenshot:



Task 12

Create the users first_user and second_user and add them to the group roles.

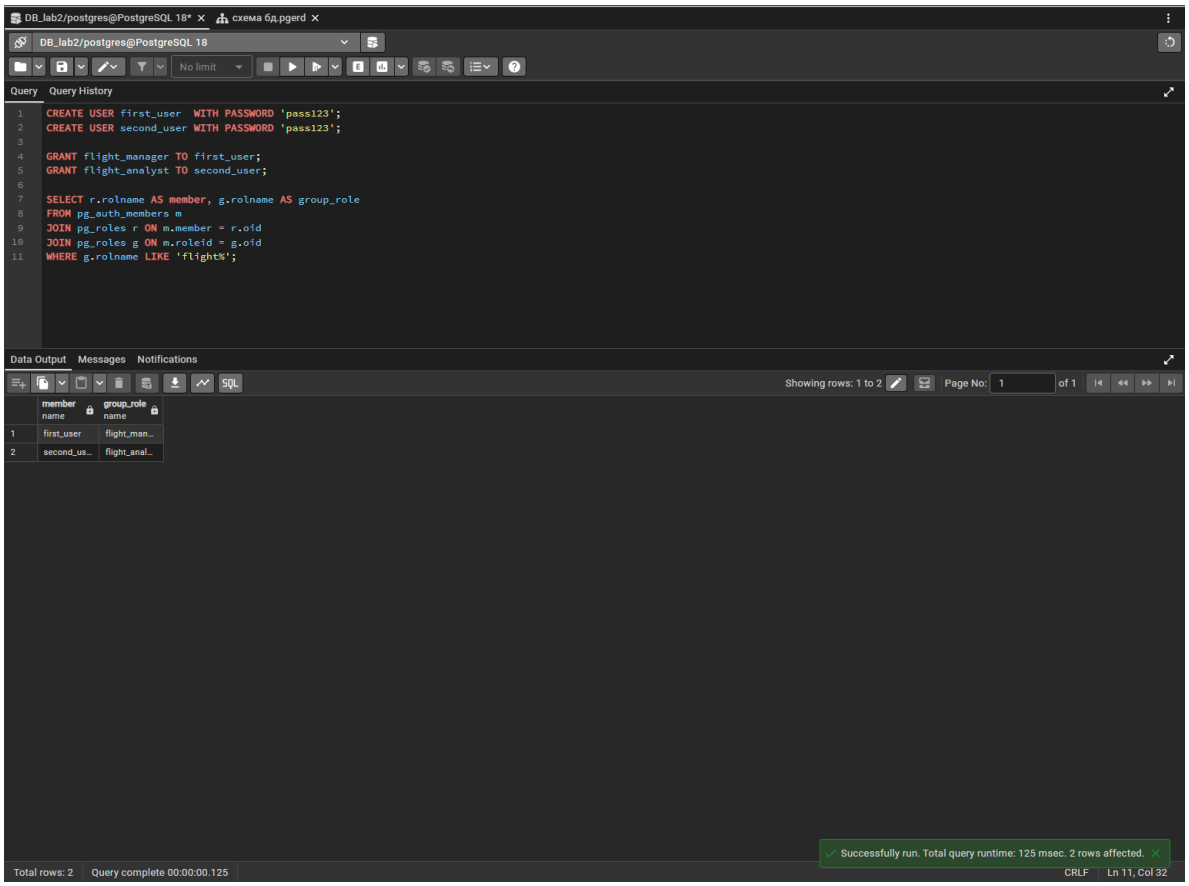
SQL code:

```
CREATE USER first_user  WITH PASSWORD 'pass123';
CREATE USER second_user WITH PASSWORD 'pass123';

GRANT flight_manager TO first_user;
GRANT flight_analyst TO second_user;

SELECT r.rolname AS member, g.rolname AS group_role
FROM pg_auth_members m
JOIN pg_roles r ON m.member = r.oid
JOIN pg_roles g ON m.roleid = g.oid
WHERE g.rolname LIKE 'flight%';
```

Screenshot:



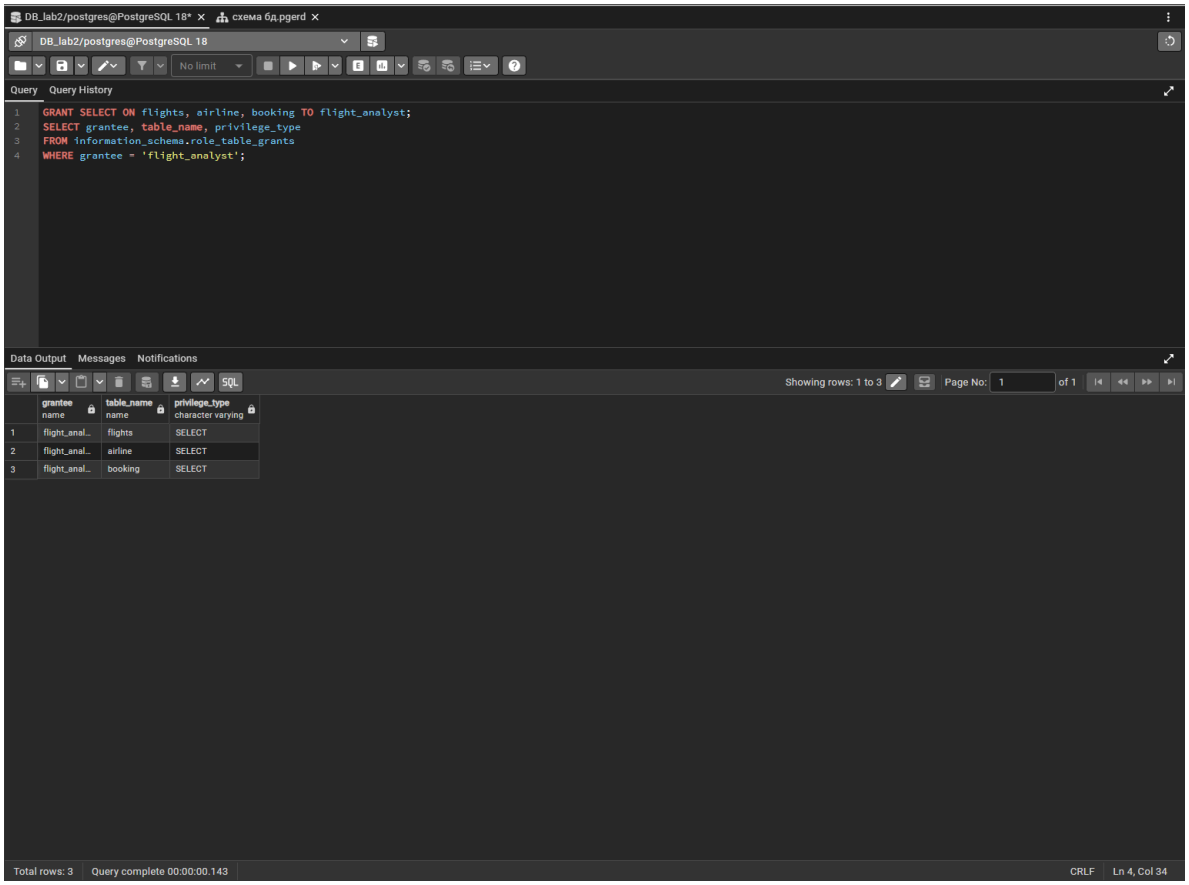
Task 13

Grant flight_analyst SELECT on the flights, airlines and bookings tables.

SQL code:

```
GRANT SELECT ON flights, airline, booking TO flight_analyst;
SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee = 'flight_analyst';
```

Screenshot:



Task 14

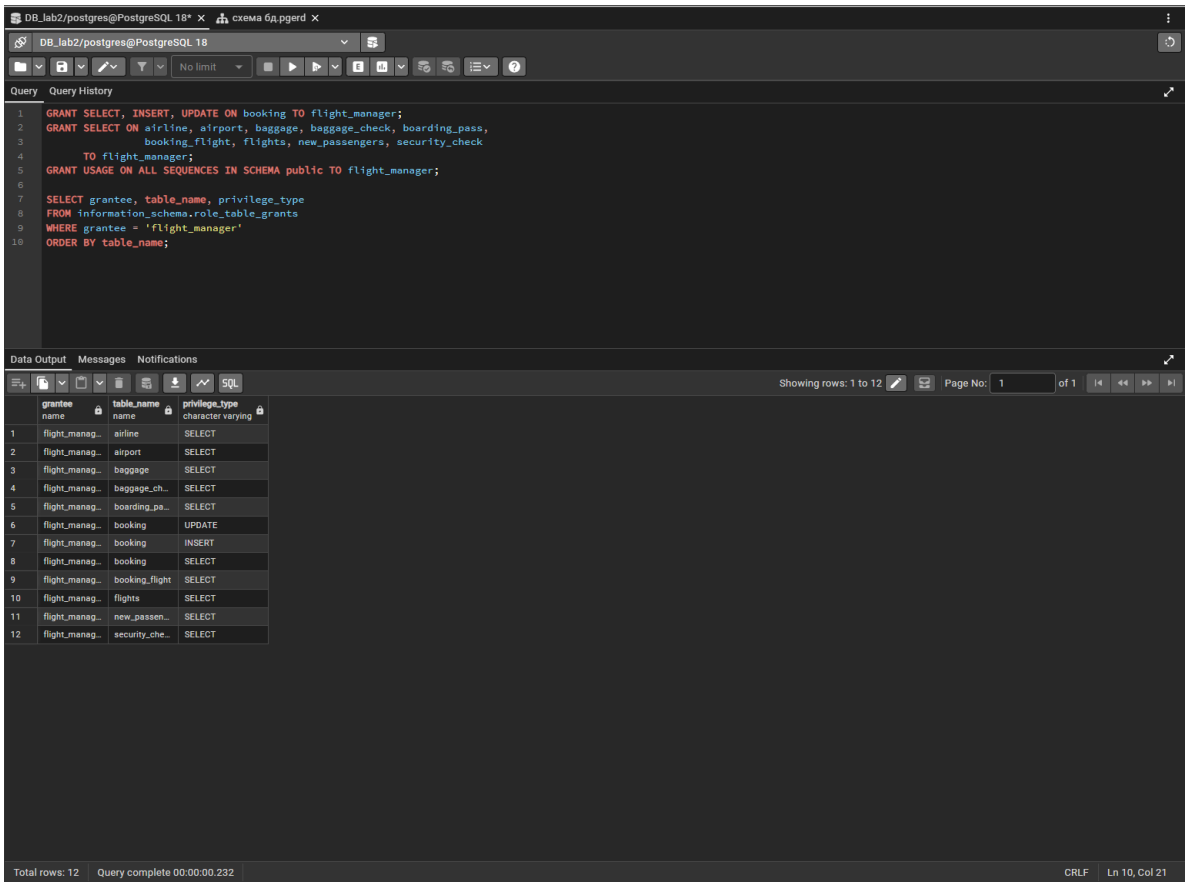
Grant flight_manager SELECT, INSERT and UPDATE on bookings, and SELECT on the other tables.

SQL code:

```
GRANT SELECT, INSERT, UPDATE ON booking TO flight_manager;
GRANT SELECT ON airline, airport, baggage, baggage_check, boarding_pass,
                booking_flight, flights, new_passengers, security_check
      TO flight_manager;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO flight_manager;

SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee = 'flight_manager'
ORDER BY table_name;
```

Screenshot:



Task 15

Connect as second_user, read flights and try to delete from bookings.

SQL code:

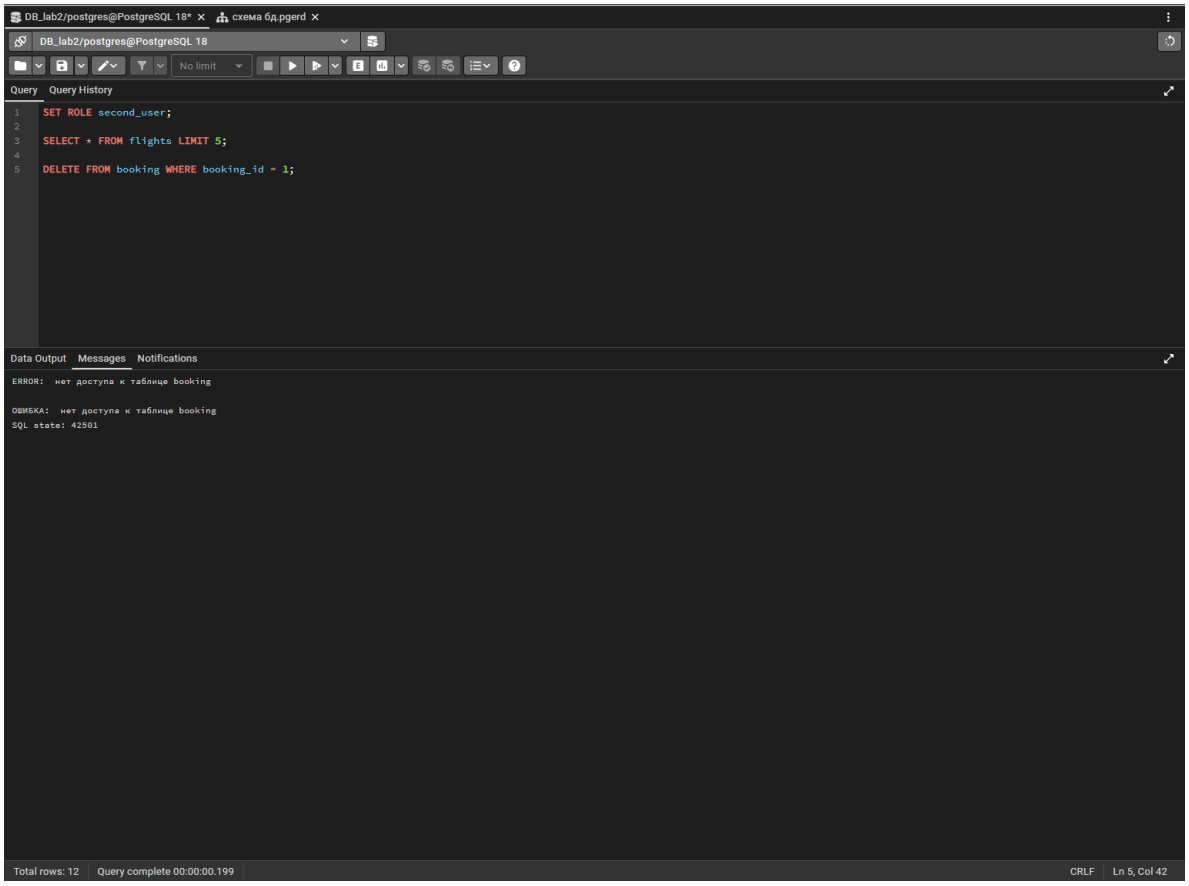
```
SET ROLE second_user;

SELECT * FROM flights LIMIT 5;

DELETE FROM booking WHERE booking_id = 1;
```

Result: second_user (flight_analyst) is not allowed to delete from booking: permission denied for table booking.

Screenshot:



Task 16

Connect as first_user, insert and update a booking, and try to delete one.

SQL code:

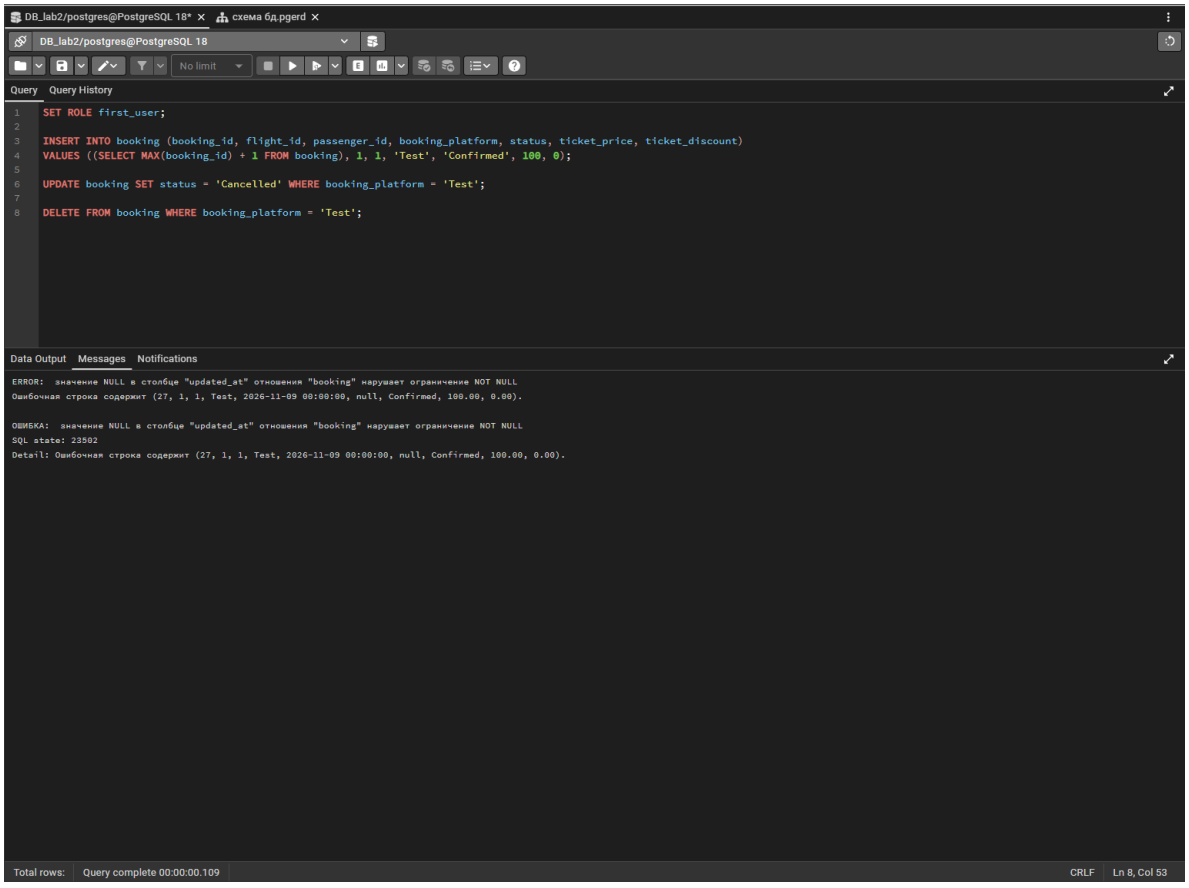
```
SET ROLE first_user;

INSERT INTO booking (booking_id, flight_id, passenger_id, booking_platform, status,
ticket_price, ticket_discount)
VALUES ((SELECT MAX(booking_id) + 1 FROM booking), 1, 1, 'Test', 'Confirmed', 100, 0);

UPDATE booking SET status = 'Cancelled' WHERE booking_platform = 'Test';

DELETE FROM booking WHERE booking_platform = 'Test';
```

Screenshot:



Task 17

Grant column-level SELECT on only the full name and country columns of passengers.

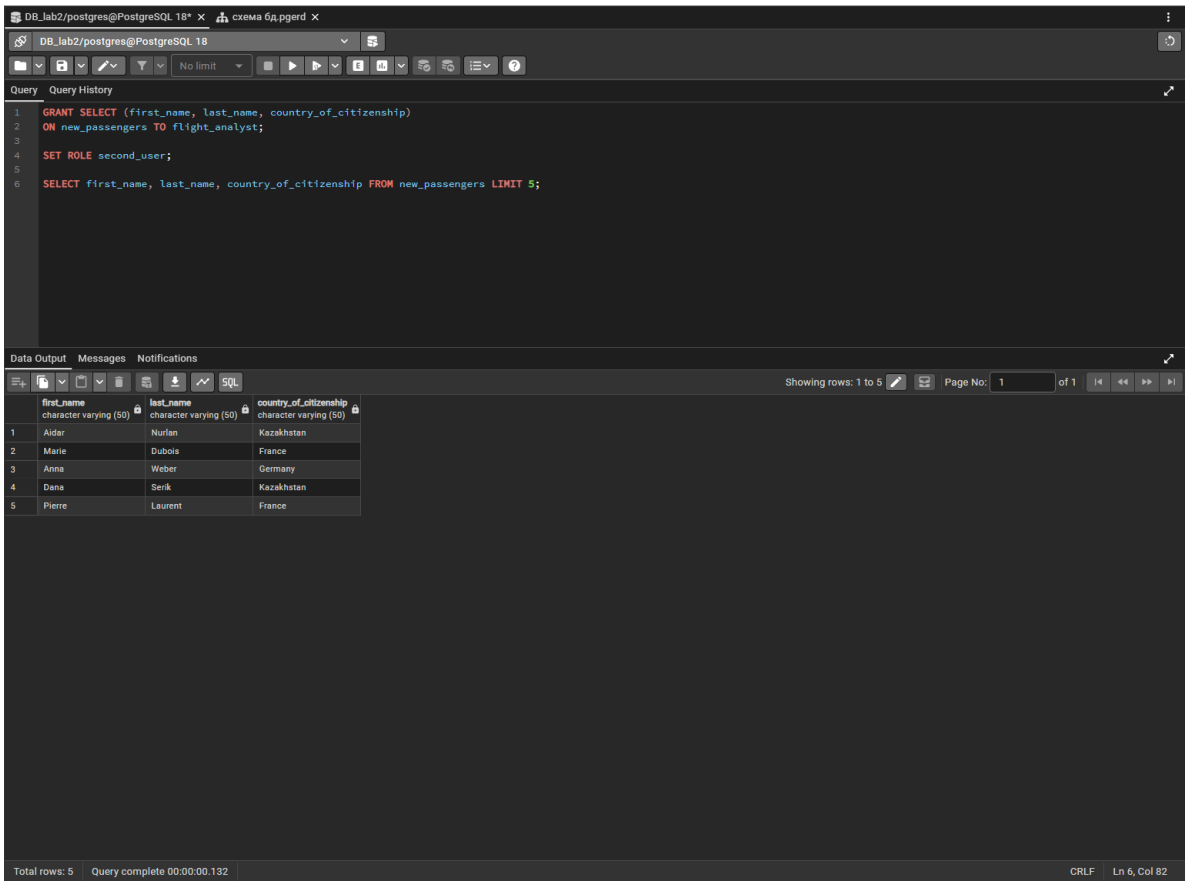
SQL code:

```
GRANT SELECT (first_name, last_name, country_of_citizenship)
ON new_passengers TO flight_analyst;

SET ROLE second_user;

SELECT first_name, last_name, country_of_citizenship FROM new_passengers LIMIT 5;
```

Screenshot:



Task 18

Revoke SELECT on bookings from flight_analyst and show that access is denied.

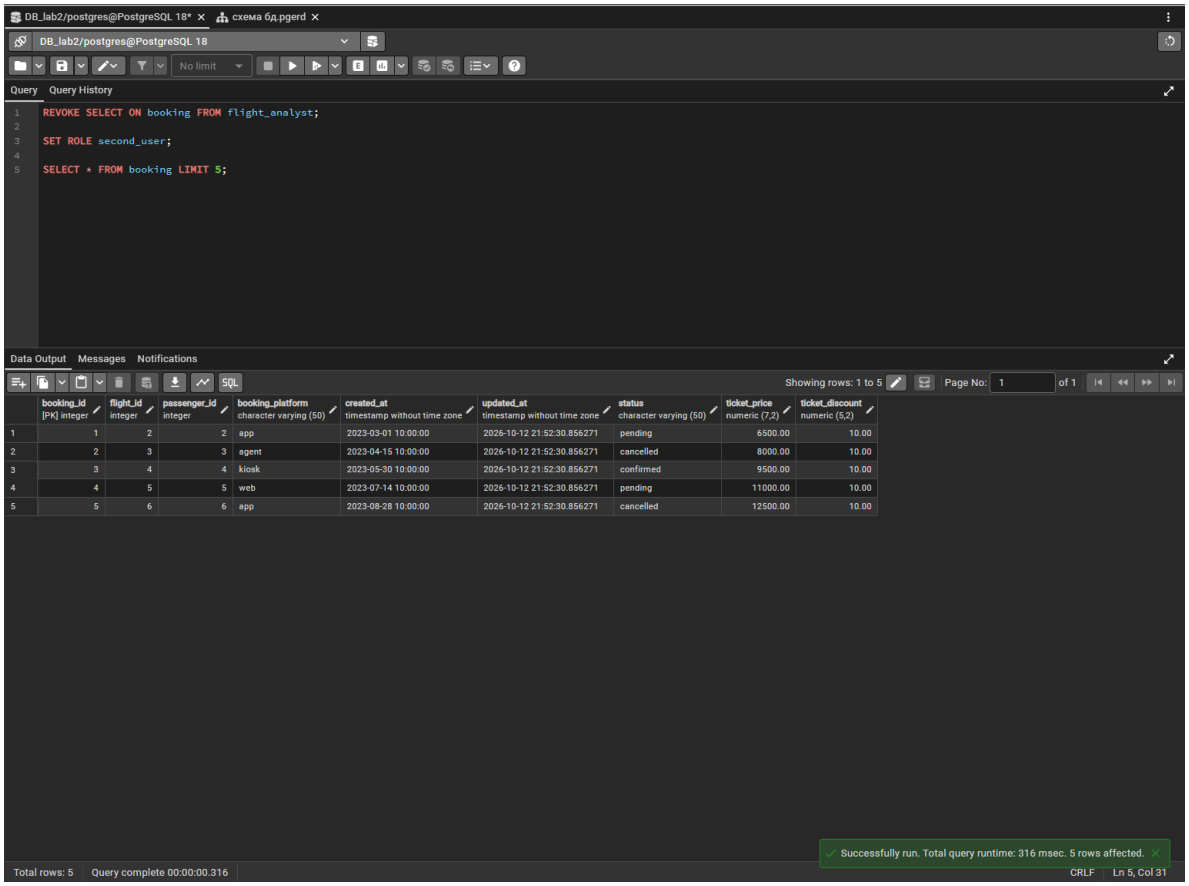
SQL code:

```
REVOKE SELECT ON booking FROM flight_analyst;

SET ROLE second_user;

SELECT * FROM booking LIMIT 5;
```

Screenshot:



Task 19

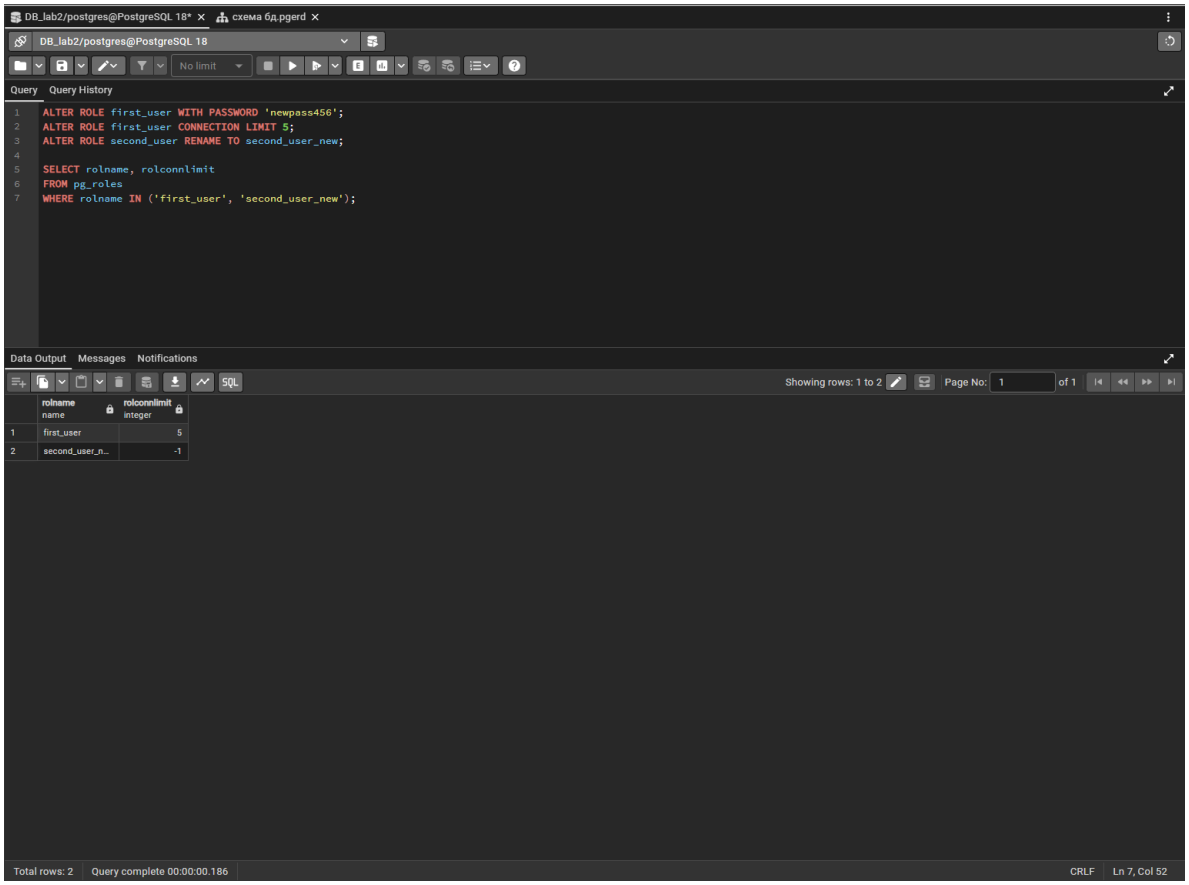
Alter the roles: change a password, set a connection limit and rename a role.

SQL code:

```
ALTER ROLE first_user WITH PASSWORD 'newpass456';
ALTER ROLE first_user CONNECTION LIMIT 5;
ALTER ROLE second_user RENAME TO second_user_new;

SELECT rolname, rolconnlimit
FROM pg_roles
WHERE rolname IN ('first_user', 'second_user_new');
```

Screenshot:



Task 20

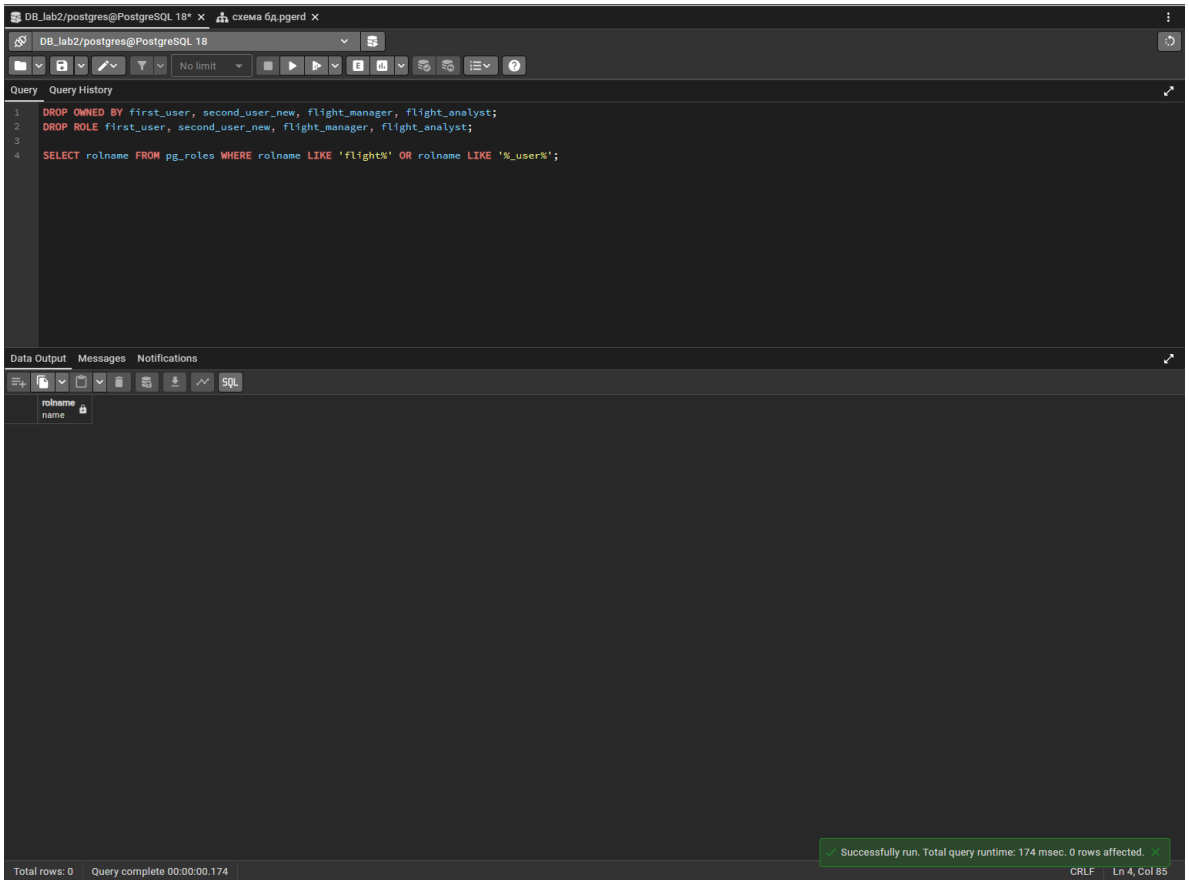
Drop all created roles.

SQL code:

```
DROP OWNED BY first_user, second_user_new, flight_manager, flight_analyst;
DROP ROLE first_user, second_user_new, flight_manager, flight_analyst;

SELECT rolname FROM pg_roles WHERE rolname LIKE 'flight%' OR rolname LIKE '%_user%';
```

Screenshot:



Conclusion

In this laboratory work I learned how to create, modify and drop views, and how to manage roles and privileges in PostgreSQL: GRANT, REVOKE, column-level access and ALTER ROLE.